

Chapter 1: Introduction

Algorithms for Optimization

2nd Edition (2026)

Kochenderfer & Wheeler

MIT Press

2026

Chapter Overview

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

A Brief History of Optimization I

Optimization is one of the oldest intellectual pursuits in mathematics. Long before computers existed, thinkers were asking: “What is the *best* arrangement, shape, or strategy?” Understanding this history helps us see why the mathematical tools we use today look the way they do.

Ancient Roots (300–250 BC)

The Greeks posed geometric extremum problems — questions such as “which shape encloses the largest area for a fixed perimeter?” **Euclid** (~300 BC) established rigorous geometric reasoning, while **Archimedes** (~250 BC) studied the problem of maximizing the area of a polygon inscribed in a circle. As the number of polygon sides grows without bound, the polygon area approaches the circle area — a conceptual forerunner of the *calculus of variations*, which optimizes over *shapes* rather than numbers. Much later, **Fibonacci** (1170–1250) popularised algebra and the idea that mathematics could be applied to solve practical problems of commerce and engineering.

A Brief History of Optimization II

The Calculus Era: 17th and 18th Centuries

The invention of calculus made it possible to study *rates of change* precisely, which turns out to be exactly what is needed to find maxima and minima.

Fermat (1643) observed that the derivative of a smooth function must equal zero at an interior maximum or minimum. This single insight — “set the derivative to zero and solve” — remains the cornerstone of optimization theory.

Newton and Leibniz (1660s–1680s) independently developed differential calculus, providing the machinery to compute derivatives systematically and to identify *stationary points* of arbitrary functions.

Euler and Lagrange (1740–1770s) extended these ideas to *functions of functions*, founding the *calculus of variations*. Lagrange also introduced his celebrated *multiplier method* for handling equality constraints — a technique still used daily in engineering and economics.

A Brief History of Optimization III

The Modern Era: Numerical and Computational Optimization

The 19th and 20th centuries brought two revolutions: the formalisation of numerical algorithms, and the arrival of computers to run them at scale.

Cauchy (1847) proposed the *steepest-descent method* — the first algorithm that iteratively improves a solution by moving in the direction of the negative gradient. This is the direct ancestor of every modern gradient-based optimizer.

Kantorovich (1939) formulated *linear programming* to solve production-planning problems efficiently. His work was so influential that he was awarded the Nobel Prize in Economics in 1975.

Dantzig (1947) invented the *Simplex algorithm*, which made linear programming computationally tractable and is still widely used today.

Kuhn and Tucker (1951) derived the *KKT conditions* — the necessary conditions for constrained optimality that generalise Fermat's derivative condition to problems with inequality and equality constraints.

The deep-learning era (2012–present): modern language models and vision systems (GPT, Llama, ResNet) are trained by minimising a loss function over *billions* of parameters using variants of stochastic gradient descent. Optimization is literally what makes AI work.

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

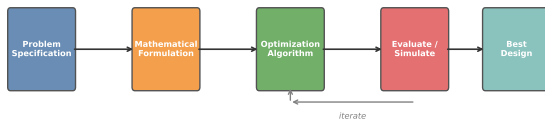
The Optimization Process I

Optimization is not just a mathematical operation — it is a *process* that begins with a real-world problem and ends with an implementable design. Understanding this process prevents the common mistake of jumping straight to an algorithm without first carefully formulating the problem.

The Five-Step Engineering Cycle

- 1 **Specify the design problem.** What physical or business system are you trying to improve? What can you actually change (the *design variables*)? What outcome do you care about (the *objective*)?
- 2 **Choose a mathematical formulation.** Translate the problem into a precise mathematical form: define the objective function $f(\mathbf{x})$, the design vector $\mathbf{x}$, and any constraints.
- 3 **Select an optimization algorithm.** Different algorithms suit different problem structures. The choice depends on the

The Optimization Process



Representative Design Problems

- **Aerospace:** choose airfoil chord, thickness, and sweep angle to minimise aerodynamic drag subject to minimum-lift and structural-integrity constraints.

The Optimization Process II

How to Choose an Algorithm

Before selecting an algorithm, you must characterise your problem along several dimensions. There is no universally best choice — the right algorithm depends on problem structure.

Dimensionality (n , the number of design variables): low-dimensional problems ($n < 10$) allow exhaustive or derivative-free search, while high-dimensional problems ($n > 1000$) typically require gradient-based methods.

Gradient availability: some problems supply exact gradients analytically or via automatic differentiation (Chapters 2–3); others only allow function evaluations (derivative-free methods, Chapter 4 onward).

Constraint structure: unconstrained problems are simpler; bound constraints (Chapter 4), equality constraints, and general inequalities (Chapter 10) each require specialised treatment.

Evaluation budget: if each evaluation of $f(\mathbf{x})$ requires an hour-long CFD simulation, you can afford far fewer iterations than if $f(\mathbf{x})$ evaluates in microseconds.

Noise and stochasticity: if $f(\mathbf{x})$ is corrupted by random noise (e.g., measured experimentally), stochastic optimizers (Chapter 8–9) are more appropriate than deterministic gradient descent.

Multi-modality: if the objective landscape has many local minima, global search strategies (genetic

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation**
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

Mathematical Formulation of an Optimization Problem I

Before an algorithm can be applied, the real-world problem must be translated into a precise mathematical statement. This section introduces the standard form used throughout the book.

Standard Form

An optimization problem is written as:

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \quad h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p$$

Mathematical Formulation of an Optimization Problem II

Notation: Every Symbol Explained

- $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top \in \mathbb{R}^n$ is the **design vector** (also called the *decision variable*). It is a column vector of n real numbers that the algorithm is free to choose. Each x_i (x sub i) is one design parameter — for example, the thickness of an airfoil at position i along the wing.
- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the **objective function**, a rule that takes a design vector $\mathbf{x}$ and returns a single real number measuring how “good” that design is. We write *min* to indicate we want the smallest possible value.
- $g_i(\mathbf{x}) \leq 0$ for $i = 1, \dots, m$ are the **inequality constraints**. There are m (m) of them. Writing $g_i \leq 0$ is a convention: constraints such as “thrust ≥ 500 N” are rewritten as $-\text{thrust} + 500 \leq 0$.
- $h_j(\mathbf{x}) = 0$ for $j = 1, \dots, p$ are the **equality constraints**. There are p (p) of them. For example, “total portfolio weight equals 1” becomes $\sum_i x_i - 1 = 0$.
- $\mathcal{X}$ (calligraphic X , read “script X ”) is the **feasible set**: the set of all $\mathbf{x}$ that satisfy every constraint simultaneously. Only points inside $\mathcal{X}$ are valid solutions.

Mathematical Formulation of an Optimization Problem III

How to Read This Formula

The expression $\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x})$ means: “find the value of $\mathbf{x}$ that lies inside the feasible set $\mathcal{X}$ and produces the smallest possible output of f .” If the feasible set is empty (the constraints are contradictory), the problem has no solution. If f can decrease without bound inside $\mathcal{X}$, the problem is *unbounded* and also has no finite solution.

Mathematical Formulation of an Optimization Problem IV

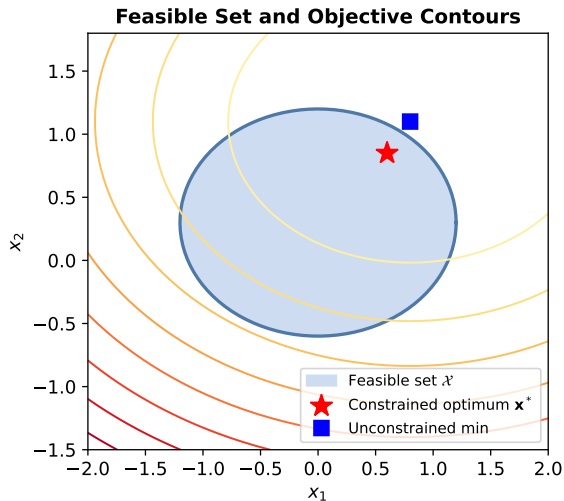
The Feasible Set in Detail

The feasible set collects every design point that respects all constraints at once:

$$\mathcal{X} = \{ \mathbf{x} \mid g_i(\mathbf{x}) \leq 0 \forall i, h_j(\mathbf{x}) = 0 \forall j \}$$

Here $\forall$ (for all) means “for every.” A point $\mathbf{x}$ is called **feasible** when it belongs to $\mathcal{X}$, and **infeasible** otherwise.

The feasible set can be a complicated geometric region — a polygon, a curved surface, or even a disconnected collection of points. The figure on the right illustrates a two-dimensional example where the feasible set is shaded.



Three Types of Constraint

Feasible Set — Worked Numerical Example I

To make the abstract formulation concrete, let us work through a small two-dimensional example by hand.

Example: A 2-D Constrained Problem

We wish to minimise the squared distance from the point $(2, 1)$:

$$\min_{x_1, x_2} f(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 1)^2$$

subject to three constraints:

$$g_1 : x_1 + x_2 \leq 2, \quad g_2 : x_1 \geq 0, \quad g_3 : x_2 \geq 0.$$

Here x_1 (x sub 1) and x_2 (x sub 2) are the two decision variables. The objective f measures the squared Euclidean distance between the current point (x_1, x_2) and the target $(2, 1)$.

Feasible Set — Worked Numerical Example II

Step 1: Find the Unconstrained Minimum

Without any constraints, we minimise f by setting it to zero. Since $f = (x_1 - 2)^2 + (x_2 - 1)^2$ is a sum of squares, it achieves its minimum value of 0 exactly at $(x_1, x_2) = (2, 1)$.

Check feasibility: does $(2, 1)$ satisfy g_1 ?

$$x_1 + x_2 = 2 + 1 = 3 \not\leq 2. \quad \text{Infeasible!}$$

The unconstrained optimum violates the first constraint, so we cannot simply ignore the constraints here.

Feasible Set — Worked Numerical Example III

Step 2: Examine Boundary Points

Because the unconstrained minimum is outside $\mathcal{X}$, the constrained minimum must lie *on the boundary* of the feasible set. We check the boundary $x_1 + x_2 = 2$ (where g_1 is active):

Substituting $x_2 = 2 - x_1$ into f :

$$f(x_1, 2 - x_1) = (x_1 - 2)^2 + (2 - x_1 - 1)^2 = (x_1 - 2)^2 + (1 - x_1)^2.$$

Expanding: $= x_1^2 - 4x_1 + 4 + x_1^2 - 2x_1 + 1 = 2x_1^2 - 6x_1 + 5$. Setting the derivative to zero:
 $4x_1 - 6 = 0 \Rightarrow x_1 = 1.5$. Then $x_2 = 2 - 1.5 = 0.5$.

$$f(1.5, 0.5) = (1.5 - 2)^2 + (0.5 - 1)^2 = 0.25 + 0.25 = 0.5.$$

Classification of Sample Points

Point	Feasible?	f value	Note
(0, 0)	Yes	$4 + 1 = 5$	Corner of $\mathcal{X}$
(1, 1)	Yes	$1 + 0 = 1$	Boundary (g_1 active)

Checking Feasibility in Python

A quick sanity check for any candidate point (x_1, x_2) :

- Compute $g_1 = x_1 + x_2 - 2$. Feasible iff $g_1 \leq 0$.

Constrained Optimisation in Python — SciPy Example

Python's `scipy.optimize.minimize` function can solve this problem automatically. The code below sets up the objective, constraints, and bounds, then calls the SLSQP solver (Sequential Least Squares Programming), which is designed specifically for smooth constrained problems.

Listing 1: Constrained optimization with SciPy

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 # Objective:  $f(x) = (x_1-2)^2 + (x_2-1)^2$ 
5 def f(x):
6     return (x[0]-2)**2 + (x[1]-1)**2
7
8 # Constraints:  $x_1+x_2 \leq 2$ ,  $x_1 \geq 0$ ,  $x_2 \geq 0$ 
9 # SciPy 'ineq' means  $\text{fun}(x) \geq 0$ , so we write
10 #  $2 - x_1 - x_2 \geq 0 \iff x_1+x_2 \leq 2$ 
11 constraints = [
12     {'type': 'ineq', 'fun': lambda x: 2 - x[0] - x
13     [1]},
14 ]
15
16 # Bounds encode  $x_1 \geq 0$  and  $x_2 \geq 0$ 
17 bounds = [(0, None), (0, None)]
18
19 x0 = np.array([0.5, 0.5]) # initial guess
20 result = minimize(f, x0,
21                  method='SLSQP',
```

Reading the SciPy API

- `method='SLSQP'`: the solver algorithm. SLSQP handles both inequality and equality constraints by solving a sequence of quadratic subproblems.
- `bounds`: a list of (lower, upper) pairs — one per variable. None means no bound on that side.
- `constraints`: a list of dictionaries.
 - `'type': 'ineq'` means $\text{fun}(\mathbf{x}) \geq 0$. Note this is the *opposite* sign from the $g_i \leq 0$ convention: you may need to negate your constraint function.

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications**
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

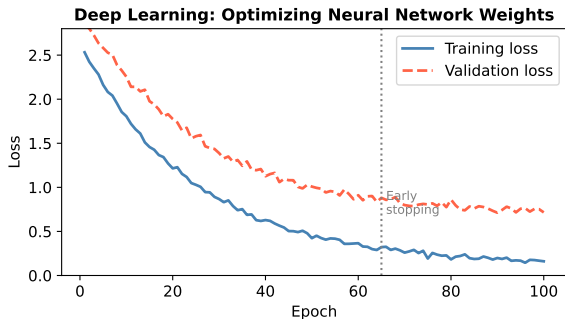
Applications of Optimization I

Optimization pervades nearly every quantitative field. The five examples below show how the same mathematical framework — choose $\mathbf{x}$ to minimise $f(\mathbf{x})$ subject to constraints — applies across very different domains.

Aircraft Design (Aeronautics)

An aircraft designer wants to minimise aerodynamic drag $D(\mathbf{x})$, which directly determines fuel consumption and operating cost. The design vector $\mathbf{x}$ collects geometric parameters of the wing and fuselage: chord length (the front-to-back distance of the wing), thickness-to-chord ratio, and sweep angle.

The constraints enforce that the wing must generate at least a minimum amount of lift to keep the aircraft airborne, and that structural stresses remain below material limits. Each function evaluation requires a Computational Fluid Dynamics (CFD) simulation that may take



The figure shows typical training and validation loss curves during neural-network optimization. The training loss (solid) decreases monotonically as the optimizer adjusts the weights. The validation loss (dashed) initially tracks the training loss, then may rise if the

Applications of Optimization II

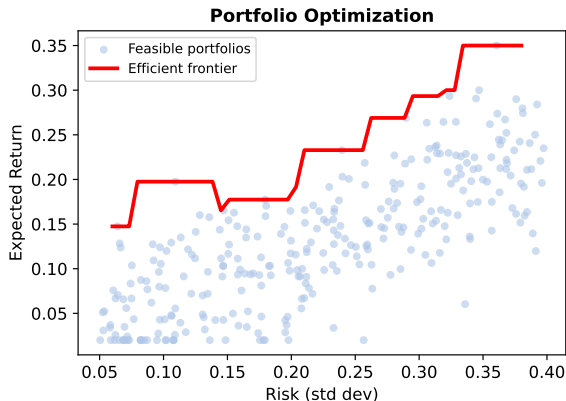
Financial Portfolio Optimization (Markowitz, 1952)

Harry Markowitz showed that portfolio selection can be cast as a quadratic program. Let $\mathbf{x} \in \mathbb{R}^n$ be the vector of portfolio weights, where x_i (x sub i) is the fraction of total wealth invested in asset i .

Objective: minimise portfolio variance (risk) $\mathbf{x}^\top \Sigma \mathbf{x}$, where Σ (Sigma, the covariance matrix) encodes how assets move together. Simultaneously, we want the expected return $\boldsymbol{\mu}^\top \mathbf{x}$ (where $\boldsymbol{\mu}$, mu, is the vector of expected returns) to exceed a target level $r_{\min}$.

Constraint: $\sum_{i=1}^n x_i = 1$ (we must invest all our wealth), and $x_i \geq 0$ (no short-selling).

By solving this for many values of $r_{\min}$ and



Applications of Optimization III

Medical Radiation Therapy Planning

In cancer radiotherapy, the goal is to deliver a prescribed radiation dose to the tumour while minimising collateral damage to surrounding healthy tissue. The optimizer chooses the *beam intensities* and *beam angles* (collected in the design vector $\mathbf{x}$).

The objective is a risk measure over the spatial dose distribution — for example, the Conditional Value-at-Risk (CVaR) of dose to critical organs. Constraints require that the tumour receives at least the prescribed minimum dose everywhere.

This is a large-scale linear or quadratic program, typically solved by interior-point methods (Chapter 11).

Cross-Domain Summary

Despite their apparent differences, all five applications share the same mathematical skeleton: choose a vector of decision variables to minimise a scalar objective subject to constraints.

Domain	Minimise	Variables
Aeronautics	Drag	Wing geometry
Deep learning	Loss	Network weights
Finance	Portfolio risk	Asset fractions
Medicine	Organ dose	Beam intensities
Manufacturing	Makespan	Job-machine assign

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global**
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

Types of Minima I

Not all “low points” of a function are equally desirable. An algorithm might converge to a point that is locally optimal but globally suboptimal. This section defines precisely what we mean by local and global minima.

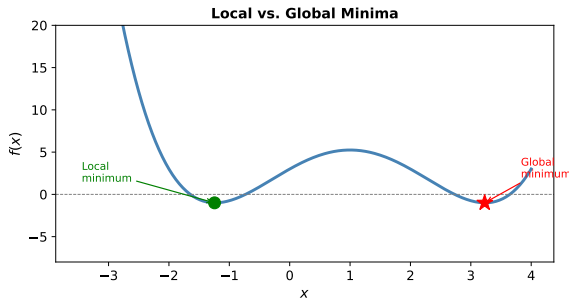
Global Minimum

A point $\mathbf{x}^*$ is a **global minimum** of f over $\mathcal{X}$ if

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \text{for all } \mathbf{x} \in \mathcal{X}.$$

In words: no other feasible point produces a smaller objective value. If the inequality is strict ($<$) for all $\mathbf{x} \neq \mathbf{x}^*$, then $\mathbf{x}^*$ is the *unique* global minimum.

Finding the global minimum of a general non-convex function is NP-hard — in the worst case, no algorithm can guarantee finding it without examining every point in $\mathcal{X}$.



Concrete Numerical Example

Consider $f(x) = \frac{x^3}{3} - x$.

Types of Minima II

Classification of Stationary Points

Type	Condition on f	Uniqueness	Remark
Strong local min	$f(\mathbf{x}^*) < f(\mathbf{x})$ for nearby $\mathbf{x} \neq \mathbf{x}^*$	Locally unique	Most algorithms find
Weak local min	$f(\mathbf{x}^*) \leq f(\mathbf{x})$ for nearby $\mathbf{x}$	Plateau possible	Flat regions complica
Global min	$f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{X}$	May or may not be unique	Hard to certify in ge
Saddle point	$f'(\mathbf{x}^*) = 0$ but not a min or max	N/A	Gradient descent can

Types of Minima III

Practical Implication: Most Algorithms Find Local Minima

The vast majority of practical optimization algorithms (gradient descent, Newton's method, quasi-Newton, conjugate gradients) are designed to find *local* minima. If the objective is non-convex, they may converge to a local minimum that is not globally optimal.

Common strategies to escape local minima and search for a global minimum:

- **Multi-start**: run the algorithm from many random initial points and keep the best solution found.
- **Simulated annealing**: occasionally accept worse solutions with a probability that decreases over time, allowing the search to “climb out” of local basins.
- **Population methods** (genetic algorithms, particle swarm): maintain a diverse population of solutions to explore multiple regions.
- **Convex relaxation**: reformulate the problem to remove non-convexities and obtain a global bound.

Exercise 1.1

Question: Give a function that has a local minimum that is not a global minimum.

Exercise 1.2

Question: What is the minimum of $f(x) = x^3 - x$?

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions**
- 7 Overview of the Book
- 8 Summary & Exercises

Optimality Conditions — One Variable I

A key question is: how do we *recognise* a minimum without exhaustively checking every point? The answer lies in optimality conditions — mathematical properties that every minimum must satisfy. We begin with the one-dimensional case for clarity, then generalise.

Setting: Unconstrained Univariate Minimisation

We consider the problem of minimising a smooth (continuously differentiable) function $f : \mathbb{R} \rightarrow \mathbb{R}$ without any constraints. Our goal is to find conditions on x^* that are *necessary* for x^* to be a local minimum.

First-Order Necessary Condition (FONC)

If x^* is a local minimum of f , then

$$f'(x^*) = 0.$$

Here $f'(x^*)$ (f prime at x -star) is the derivative of f evaluated at x^* . A point where the derivative is zero is called a **stationary point**.

Sufficient Condition for a Strong Local Minimum

If both of the following hold:

$$f'(x^*) = 0 \quad \text{and} \quad f''(x^*) > 0,$$

then x^* is a **strong local minimum**. The strict inequality $f''(x^*) > 0$ ensures that the function curves upward, guaranteeing that nearby points^{28 / 49}

Optimality Conditions — One Variable II

Derivation via Taylor Expansion

To see *why* the FONC holds, we use a Taylor expansion. For a small step $h > 0$ (h , a positive real number), we have:

$$f(x^* + h) = f(x^*) + h f'(x^*) + O(h^2), \quad (1.4)$$

$$f(x^* - h) = f(x^*) - h f'(x^*) + O(h^2). \quad (1.5)$$

Here $O(h^2)$ (“big-O of h-squared”) denotes terms that shrink faster than h as $h \rightarrow 0$ and can be neglected for small h .

If x^* is a local minimum, then for small enough h :

$$f(x^* + h) \geq f(x^*) \implies h f'(x^*) \geq 0, \quad (1.6)$$

$$f(x^* - h) \geq f(x^*) \implies h f'(x^*) \leq 0. \quad (1.7)$$

Combining (1.6) and (1.7): $h f'(x^*)$ must be both ≥ 0 and ≤ 0 simultaneously, which forces $f'(x^*) = 0$.

Optimality Conditions — One Variable III

Derivation of the SONC via Taylor Expansion

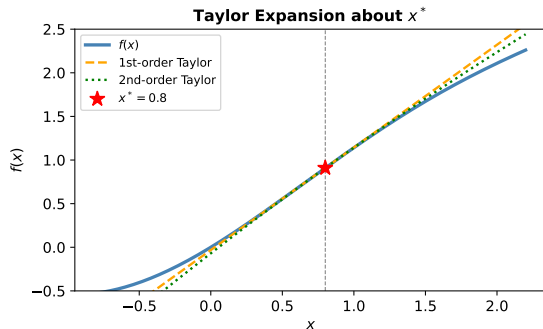
Using the FONC ($f'(x^*) = 0$), the second-order Taylor expansion gives:

$$f(x^* + h) = f(x^*) + \underbrace{h f'(x^*)}_{=0} + \frac{h^2}{2} f''(x^*) + O(h^3). \quad (1.9)$$

If x^* is a local minimum, $f(x^* + h) \geq f(x^*)$ for small h , so:

$$\frac{h^2}{2} f''(x^*) \geq 0 \implies f''(x^*) \geq 0, \quad (1.10)$$

since $h^2 > 0$ for $h \neq 0$.



The Taylor approximation

$f(x^* + h) \approx f(x^*) + hf'(x^*) + \frac{h^2}{2} f''(x^*)$ becomes exact as $h \rightarrow 0$. The figure illustrates how well successive orders of the Taylor polynomial approximate f near x^* .

Optimality Conditions — One Variable IV

Worked Example: Applying the Conditions to $f(x) = x^4 - 4x^2$

Let us verify the optimality conditions at the local minima of $f(x) = x^4 - 4x^2$.

Step 1 — Find stationary points (FONC):

$$f'(x) = 4x^3 - 8x = 4x(x^2 - 2) = 0.$$

Solutions: $x = 0$, $x = \sqrt{2} \approx 1.414$, $x = -\sqrt{2} \approx -1.414$.

Step 2 — Apply the SONC at each stationary point:

$$f''(x) = 12x^2 - 8.$$

- At $x = 0$: $f''(0) = -8 < 0$ — this is a **local maximum**.
- At $x = \sqrt{2}$: $f''(\sqrt{2}) = 12 \cdot 2 - 8 = 16 > 0$ — **strong local minimum**, $f(\sqrt{2}) = 4 - 8 = -4$.
- At $x = -\sqrt{2}$: $f''(-\sqrt{2}) = 16 > 0$ — **strong local minimum**, $f(-\sqrt{2}) = -4$.

Since $f(x) \rightarrow +\infty$ as $x \rightarrow \pm\infty$, both points $x = \pm\sqrt{2}$ are also **global minima** with equal minimum

Optimality Conditions — Multiple Variables I

The one-dimensional conditions extend naturally to functions of n variables, but the derivative is replaced by the *gradient* and the second derivative is replaced by the *Hessian matrix*. These objects are central to the rest of the book.

First-Order Necessary Condition (Multivariate FONC)

If $\mathbf{x}^*$ is a local minimum of $f : \mathbb{R}^n \rightarrow \mathbb{R}$, then

$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$

Here ∇ (nabla, also written ∇ , meaning “gradient” or “del”) denotes the vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^\top.$$

The notation $\partial f / \partial x_i$ (partial derivative of f with respect to x_i) measures how f changes when only x_i

Second-Order Necessary Condition (Multivariate SONC)

If $\mathbf{x}^*$ is a local minimum, then the **Hessian matrix** $\nabla^2 f(\mathbf{x}^*)$ must be *positive semidefinite*:

$$\nabla^2 f(\mathbf{x}^*) \succeq 0.$$

The symbol $\succeq 0$ (succeeds or equals zero) means positive semidefinite: every eigenvalue is ≥ 0 . The Hessian is the $n \times n$ matrix of second-order partial derivatives:

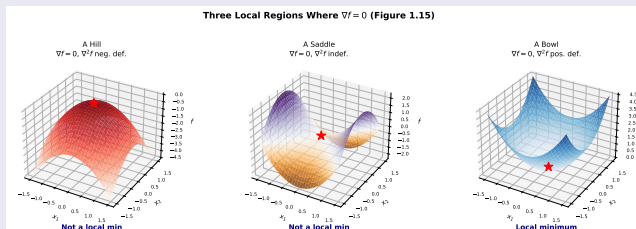
$$[\nabla^2 f(\mathbf{x})]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

Optimality Conditions — Multiple Variables II

Three Cases at a Stationary Point ($\nabla f = 0$)

Once we find a stationary point $\mathbf{x}^*$ (satisfying FONC), the Hessian tells us what kind of stationary point it is:

- 1 **Local minimum (bowl shape):** $\nabla^2 f(\mathbf{x}^*) \succ 0$ (positive definite — all eigenvalues > 0). The function curves upward in every direction.
- 2 **Local maximum (hill shape):** $\nabla^2 f(\mathbf{x}^*) \prec 0$ (negative definite — all eigenvalues < 0). The function curves downward in every direction.
- 3 **Saddle point:** $\nabla^2 f(\mathbf{x}^*)$ is indefinite (some eigenvalues positive, some negative). The function curves upward in some directions and downward in others — like the surface of a horse saddle. This is *not* a local minimum.



Optimality Conditions — Python Verification

The following Python code computes the gradient and Hessian of the Rosenbrock function numerically and verifies that $(1, 1)$ satisfies both FONC and SONC. This pattern — compute gradient, compute Hessian, check eigenvalues — is a standard verification workflow.

Listing 2: Checking optimality conditions for the Rosenbrock function

```
1 import numpy as np
2
3 def rosenbrock(x):
4     """f(x) = (1-x1)^2 + 5*(x2-x1^2)^2"""
5     return (1 - x[0])**2 + 5*(x[1] - x[0]**2)**2
6
7 def grad_rosenbrock(x):
8     """Gradient vector [df/dx1, df/dx2]"""
9     x1, x2 = x
10    df_dx1 = 2*(10*x1**3 - 10*x1*x2 + x1 - 1)
11    df_dx2 = 10*(x2 - x1**2)
12    return np.array([df_dx1, df_dx2])
13
14 def hess_rosenbrock(x):
15     """2x2 Hessian matrix of second derivatives"""
16    x1, x2 = x
17    h11 = -20*(x2 - x1**2) + 40*x1**2 + 2
18    h12 = -20*x1
19    h22 = 10
```

Expected Output

Gradient: [0. 0.]

Hessian:
[[42. -20.]
 [-20. 10.]]

Eigenvalues: [2.0, 50.0]
(both positive)

FONC satisfied: True

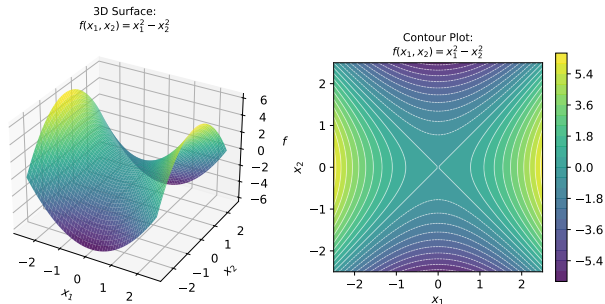
SONC satisfied: True

Conclusion

Both conditions are satisfied at $\mathbf{x}^* = (1, 1)$, confirming it is a strong local minimum.

Surface and Contour Visualisation

Visualising the objective function as a 3-D surface and a 2-D contour plot helps develop geometric intuition about where minima, maxima, and saddle points are located. The code below plots the saddle function $f(x_1, x_2) = x_1^2 - x_2^2$.



Listing 3: 3D surface + contour plot

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
5 x1 = np.linspace(-2.5, 2.5, 100)
6 x2 = np.linspace(-2.5, 2.5, 100)
7 X1, X2 = np.meshgrid(x1, x2)
8 Z = X1**2 - X2**2 # saddle function
9
10 fig = plt.figure(figsize=(10, 4))
11
12 # 3-D surface plot
13 ax3d = fig.add_subplot(121, projection='
14     3d')
15 ax3d.plot_surface(X1, X2, Z, cmap='
16     viridis')
17 ax3d.set_xlabel('$x_1$')
18 ax3d.set_ylabel('$x_2$')
19 ax3d.set_title('Surface')
```

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises

Overview of the Book I

This book is organised around the structure of optimization problems: we begin with the simplest case (one-dimensional, unconstrained, smooth) and progressively add complexity (higher dimensions, constraints, noise, multiple objectives). Understanding where each chapter fits in the overall roadmap will help you navigate the material.

Part I: Foundations (Chapters 1–4)

- Ch. 1 Introduction** (this chapter): formulation, history, applications, and optimality conditions.
- Ch. 2 Derivatives and Gradients**: how to compute derivatives analytically, numerically (finite differences), and exactly (automatic differentiation).
- Ch. 3 Bracketing**: finding an interval that contains a minimum for one-dimensional problems.
- Ch. 4 Local Descent**: the general descent

Part III: Constrained and Specialised (Chapters 10–16)

- Ch. 10 Constraints**: KKT conditions, penalty methods, augmented Lagrangian, active-set methods.
- Ch. 11 Linear Programming**: Simplex algorithm, interior-point methods.
- Ch. 12 Network Flow**: min-cost flow, shortest paths.
- Ch. 13 Sampling Plans**: space-filling designs for expensive black-box functions.

Overview of the Book II

The Progression from Simple to Complex

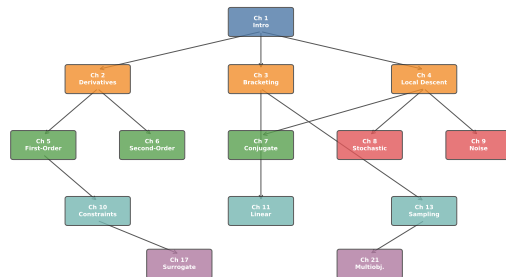
Chapters 5–9 all assume the objective is smooth and unconstrained (or only box-constrained). They differ in *what information about f they use*:

Chapter 5 uses only the gradient ∇f (first-order information). This is sufficient to find a descent direction, but convergence can be slow near the minimum because the gradient gives no information about curvature.

Chapter 6 adds the Hessian $\nabla^2 f$ (second-order information), allowing Newton's method to take much more accurate steps. The cost is computing and factorising the $n \times n$ Hessian, which is expensive when n is large.

Chapter 7 uses conjugate directions, which accumulate information from previous steps to

Chapter Dependency Overview (simplified)



The chapter dependency graph shows which earlier chapters are prerequisites for each later chapter. Arrows indicate “should be read before.” Chapters 1–4 form the mandatory foundation for all subsequent material.

When Gradients Are Unavailable: Surrogate and Bayesian Methods

Many real engineering problems do not provide gradients. Either the objective is computed by a legacy simulator that outputs only a scalar, or the function is stochastic and gradients are too noisy to be useful. In these cases, we must rely on *function values alone*.

Surrogate models (Chapter 14): fit a computationally cheap model (such as a Gaussian process or polynomial regression) to the observed function values, then optimise the cheap model instead.

Bayesian optimisation (Chapter 17): maintain a probabilistic belief (posterior distribution) over the function f , and choose the next evaluation point by maximising an *acquisition function* that balances exploitation (evaluating near the current best estimate) with exploration (reducing uncertainty in unexplored regions).

Overview of the Book IV

Algorithm Families at a Glance

Family	Information used	Key chapter(s)	Typical n
First-order gradient	∇f	5, 6	Any
Second-order (Newton)	$\nabla^2 f$	6	Small–medium
Conjugate direction	History of ∇f	7	Medium
Stochastic gradient	Noisy ∇f	8, 9	Very large
Evolutionary	f values only	14	Small–medium
Bayesian optimisation	Posterior over f	17	Very small

Outline

- 1 History of Optimization
- 2 The Optimization Process
- 3 Mathematical Formulation
- 4 Applications
- 5 Minima: Local vs. Global
- 6 Optimality Conditions
- 7 Overview of the Book
- 8 Summary & Exercises**

Summary of Chapter 1 I

Chapter 1 introduced the fundamental concepts that underpin every topic in this book. Before proceeding to the algorithms, make sure you can state and apply each of the following ideas.

Core Concepts

① **What is optimization?** The process of choosing a design vector $\mathbf{x}$ from a feasible set $\mathcal{X}$ to minimise (or maximise) an objective function $f(\mathbf{x})$.

② **Standard form:** every problem can be written as

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{subject to constraints.}$$

③ **Local minimum:** a point where $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all nearby $\mathbf{x}$. Global minimum: this holds for *all* feasible $\mathbf{x}$.

④ **First-Order Necessary Condition (FONC):** $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Zero gradient is necessary but not sufficient for a minimum

Core Formulas to Memorise

$$\text{FONC (1-D): } f'(x^*) = 0$$

$$\text{FONC (n-D): } \nabla f(\mathbf{x}^*) = \mathbf{0}$$

$$\text{SONC (1-D): } f''(x^*) \geq 0$$

$$\text{SONC (n-D): } \nabla^2 f(\mathbf{x}^*) \succeq \mathbf{0}$$

$$\text{Sufficient: } f'(x^*) = 0, f''(x^*) > 0$$

Applications Span Many Fields

The same mathematical framework applies to: aerospace engineering (aerodynamic drag minimisation), machine learning (loss minimisation over neural network weights)

Exercises (with Full Solutions) I

The following exercises are drawn from the textbook. Work through them yourself before reading the solutions — the process of attempting a problem is more valuable than simply reading the answer.

Exercise 1.1

Question: Give an example of a function that has a local minimum that is not a global minimum.

Solution: Consider $f(x) = \frac{x^3}{3} - x$.

- $f'(x) = x^2 - 1 = 0$ gives stationary points at $x = \pm 1$.
- $f''(1) = 2 > 0$, confirming $x = 1$ is a local minimum with $f(1) = \frac{1}{3} - 1 = -\frac{2}{3}$.
- As $x \rightarrow -\infty$, $f(x) = \frac{x^3}{3} - x \rightarrow -\infty$, so the function is unbounded below. Therefore $x = 1$ cannot be a global minimum.

Exercises (with Full Solutions) II

Exercise 1.2

Question: What is the minimum of $f(x) = x^3 - x$?

Solution: No minimum exists. As $x \rightarrow -\infty$, x^3 dominates and $f(x) \rightarrow -\infty$, so f has no lower bound and therefore no global minimiser.

Exercise 1.3

Question: Give a function that is bounded below but has no global optimum on $\mathcal{X} = [0, 1]$.

Solution: Define

$$f(x) = \begin{cases} |x| & \text{if } x \neq 0 \\ 1 & \text{if } x = 0 \end{cases}.$$

This function satisfies $f(x) \geq 0$ for all x (bounded below by 0). The infimum is $\inf_{x \in [0, 1]} f(x) = 0$, but this infimum is not attained: at $x = 0$ the function value is 1, not 0, and for any $x > 0$ we have $f(x) = x > 0$. So there is no point in $[0, 1]$ where f achieves its infimum.

Exercises (with Full Solutions) III

Exercise 1.4

Question: What are the minima and minimizers of $f(x) = \sin(x)$?

Solution: The function $\sin(x)$ is periodic with period 2π (2 pi, approximately 6.28). Its minimum value is -1 , attained whenever $x = \frac{3}{2}\pi + 2\pi k$ for any integer k (since $\sin(3\pi/2) = -1$). There are infinitely many global minimizers, all equally optimal.

Exercise 1.5

Question: Does the FONC $f'(x) = 0$ necessarily hold at the optimal solution of a *constrained* problem?

Solution: No. Consider $\min_x f(x) = x$ subject to $x \geq 1$. The optimal solution is $x^* = 1$ (the smallest feasible value), but $f'(1) = 1 \neq 0$.

This shows that for constrained problems, the unconstrained FONC no longer applies. The correct conditions are the **KKT (Karush–Kuhn–Tucker) conditions** (Chapter 10), which account for the fact that the optimizer is “pushing against” a constraint boundary.

Exercises (with Full Solutions) IV

Exercise 1.6

Question: How many minima does $f(x, y) = x^2 + y$ have, subject to the constraints $x > y \geq 1$?

Solution: None. The constraint $x > y$ is a *strict* inequality, so the boundary $x = y$ is excluded from the feasible set. As we let $x \rightarrow y^+$ (x approaches y from above) and $y \rightarrow 1^+$ (y approaches 1 from above), $f(x, y) \rightarrow 1 + 1 = 2$, but we can never actually attain $f = 2$ because the boundary is excluded. Therefore the infimum (greatest lower bound) of f over the feasible set is 2, but no minimiser exists.

Exercises (with Full Solutions) V

Exercise 1.7

Question: How many inflection points does $f(x) = x^3 - 10$ have?

Solution: An inflection point is a point where the second derivative changes sign. Computing:

$$f'(x) = 3x^2, \quad f''(x) = 6x.$$

Setting $f''(x) = 6x = 0$ gives $x = 0$. For $x < 0$, $f''(x) < 0$ (concave down); for $x > 0$, $f''(x) > 0$ (concave up). Since the sign of f'' changes at $x = 0$, there is exactly **one** inflection point, at $x = 0$.

Note: the -10 is just a vertical shift and does not affect the shape or the location of the inflection point.

Exercises (with Full Solutions) VI

Study Tips for Chapter 1

- Always check *both* FONC *and* SONC when identifying a minimum. FONC alone is not sufficient — a stationary point could be a maximum or saddle.
- For constrained problems, the unconstrained FONC does not apply. The correct framework is the KKT conditions, introduced in Chapter 10. Keep this in mind when working on applied problems.
- In Python, `scipy.optimize.minimize` is your go-to solver. Use `method='L-BFGS-B'` for unconstrained or box-constrained problems, and `method='SLSQP'` when you have general inequality or equality constraints.
- Draw the function before running an algorithm. For one- and two-dimensional problems, plotting the objective and feasible set often reveals whether local minima exist and where the global minimum is likely to be.

Chapter 1: Introduction

Algorithms for Optimization

Kochenderfer & Wheeler, 2nd Ed. (2026)

What We Covered in This Chapter

- A brief history of optimization from ancient geometry to modern deep learning.
- The five-step engineering optimization process.
- Standard mathematical formulation: objective, constraints, feasible set.
- Applications across aerospace, machine learning, finance, medicine, and

Up Next: Chapter 2 — Derivatives and Gradients

Before we can use gradient-based algorithms, we need reliable methods to compute derivatives.

Chapter 2 covers:

- Finite difference approximations (forward, central, backward).
- The complex-step method for high-accuracy